视觉伺服控制的基础方法笔记

欧阳志华

日期：2020 年 4 月 16 日

1 视觉伺服误差构建

一般性的视觉伺服误差定义为：

$$e(x) = s(m(t), a) - s^* \tag{1}$$

其中 s 为视觉特征，$m(t)$ 为特征点的图像坐标，a 为相机内部参数或 3D 模型。

视觉伺服控制方法的区别都在于对 s 的设计，一旦 S 确定，控制方法就很容易得到了。最简单直接的方法就是直接设计一个速度控制器。记相机的空间速度为：$v = (v_c, \omega_c)$，则有：

$$\dot{s} = L_s v \tag{2}$$

其中 L_s 为交互矩阵或特征雅可比矩阵。由(1)和(2)可以得到相机速度和误差之间的关系:

$$\dot{e} = L_e v \tag{3}$$

其中 $L_s = L_e$。

2 控制律

假设我们希望误差 e 能够以指数形式减小即：

$$\dot{e} = \lambda e \tag{4}$$

可得：

$$v_c = -\lambda L_e^+ \tag{5}$$

其中 L_e^+ 是 L_e 的广义逆矩阵或伪逆。L_e^+ 或 L_e 一般无法直接得到因此须设计逼近器估计 L_e^+，记为 $\hat{L_e^+}$。实际的控制律即为：

$$v_c = -\lambda \hat{L_e^+} \tag{6}$$

2.1 交互矩阵估计器

有三种常见的估计器形式：

$$\hat{L_e^+} = L_{e^*}^+ \tag{7}$$

$$\hat{L_e^+} = L_e^+ \tag{8}$$

$$\hat{L_e^+} = 1/2(L_{e^*} + L_e)^+ \tag{9}$$

$L_{e^*}^+$ 是期望点的交互矩阵，L_e^+ 是当前点的交互矩阵。

3 matlab 仿真

3.1 IBVS

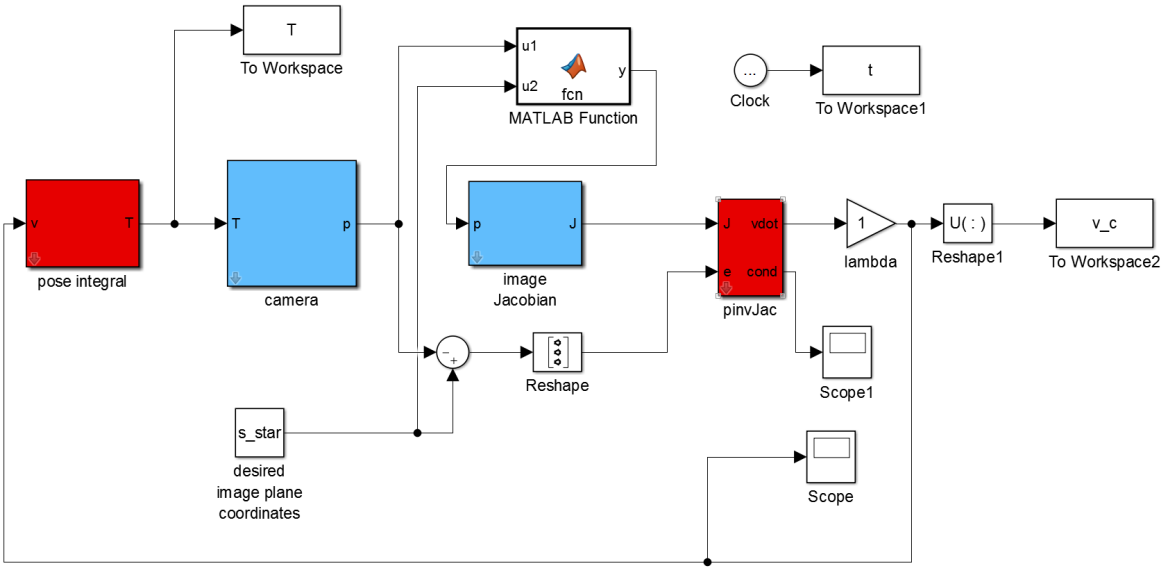

图 1: ibvs 仿真模型

camera 模块：将世界点坐标投影到图像平面

pose integer 模块：更新相机的位姿

运行代码：

```
%IBVS
clear
cam = CentralCamera('default'); %定义一个默认相机
P = mkgrid( 2, 0.5, 'T', transl(0,0,3) ); %形成边长为0.5的正方形，中心
    在（0，0，3）
s_star = bsxfun(@plus, 100*[-1 -1 1 1; -1 1 1 -1], cam.pp'); %目标特
    征，中心在主点，400*400像素正方形
cam_ip = transl(1,1,-3) * trotz(0.6);      %相机初始位姿
depth = 3;       %深度
```

```
sim('viserv_ibvs');     %运行仿真
camtraj = transl(T);     %将齐次矩阵转换为坐标
camrow = tr2rpy(T);      %将齐次矩阵转化为对应的横滚俯仰偏航角

%画图
figure(2);
plot3(camtraj(:,1),camtraj(:,2),camtraj(:,3));    %绘制相机运动轨迹
xlabel('X轴');ylabel('Y轴');zlabel('Z轴');
title('相机轨迹');
text(1,1,-3,'始点');
grid on          %打开坐标网格
figure(3);
plot(t,camrow(:,1),'-b','linewidth',1);        %绘制相机旋转角度变化
hold on
plot(t,camrow(:,2),'*g','linewidth',1);
hold on
plot(t,camrow(:,3),'-.r','linewidth',1);
xlabel('time(s)');
ylabel('横滚俯仰偏航角');
legend('R', 'P', 'Y');       %添加图例
figure(4);
plot(t,v_c,'linewidth',1);        %绘制相机速度
xlabel('time(s)');
ylabel('相机速度');
legend('v_{x}', 'v_{y}', 'v_{z}','\omega_{x}', '\omega_{y}', '\omega_{z
    }');       %添加图例,使用tex格式显示公式
```

分别对前面三种不同的交互矩阵估计器进行仿真：

3.2 PBVS

pose estimation 模块：对相机的位姿进行估计

control law 模块：速度控制器, 表达式为(10)和(11)

$$v_c = -\lambda((^{c^*}t_o - ^{c}t_o) + [^{c}t_o]_\times \theta u) \tag{10}$$

$$\omega_c = -\lambda \theta u \tag{11}$$

运行代码：

```
% PBVS控制
clear
cam = CentralCamera('default');%定义一个参数已知的相机
P = mkgrid( 2, 0.5, 'T', transl(0,0,3) ); %目标有四点正方形
T_star = transl(0, 0, 0.5);%期望的相对相机的目标位姿, 目标位于相机前方1
```

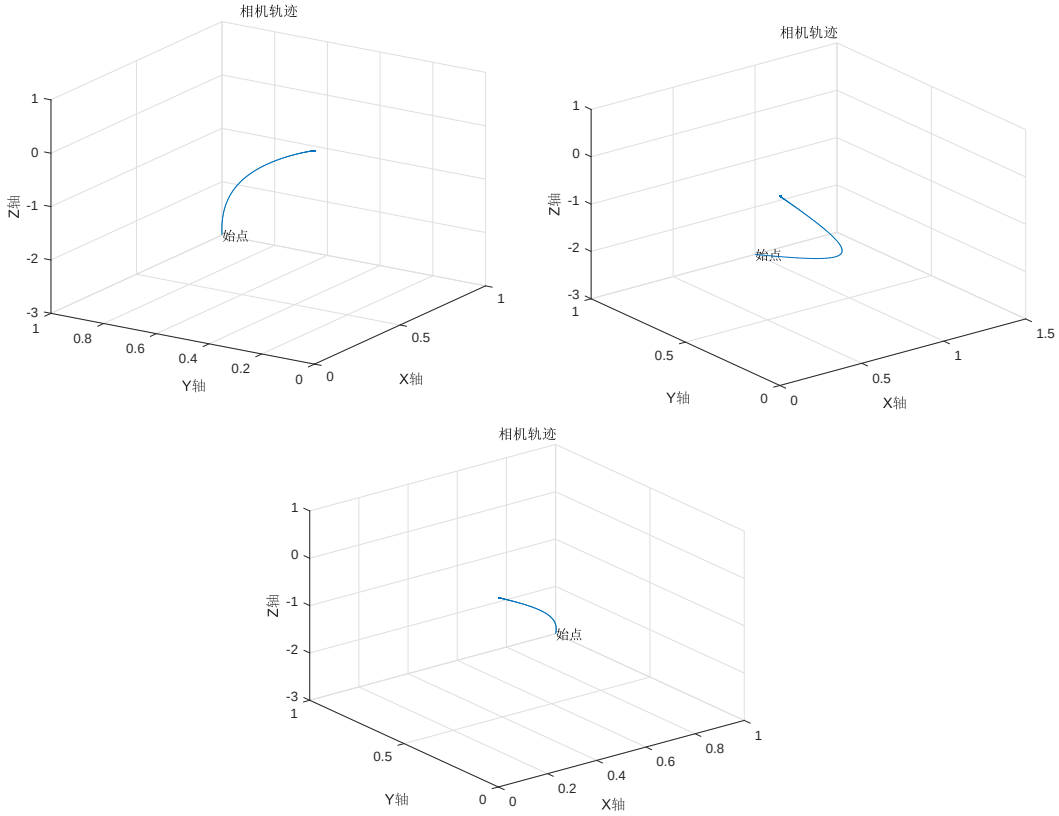

图 2: 相机轨迹

```
    米处
cam_ip = transl(1,1,-3)*trotz(0.6);%相机初始位姿
lambda = -1;

viserv_pbvs
fig = get_param('viserv_pbvs','Handle');
saveas(fig,'pbvs_model.png');    %保存模型图
sim('viserv_pbvs');
camtraj = transl(T);     %将齐次矩阵转换为坐标
camrow = tr2rpy(T);      %将齐次矩阵转化为对应的横滚俯仰偏航角

%画图
figure(2);
plot3(camtraj(:,1),camtraj(:,2),camtraj(:,3));    %绘制相机运动轨迹
xlabel('X轴');ylabel('Y轴');zlabel('Z轴');
title('相机轨迹');
text(1,1,-3,'始点');
grid on          %打开坐标网格
saveas(gcf,'traj1_pbvs','epsc');
figure(3);
plot(t,camrow(:,1),'-b','linewidth',1);        %绘制相机旋转角度变化
```

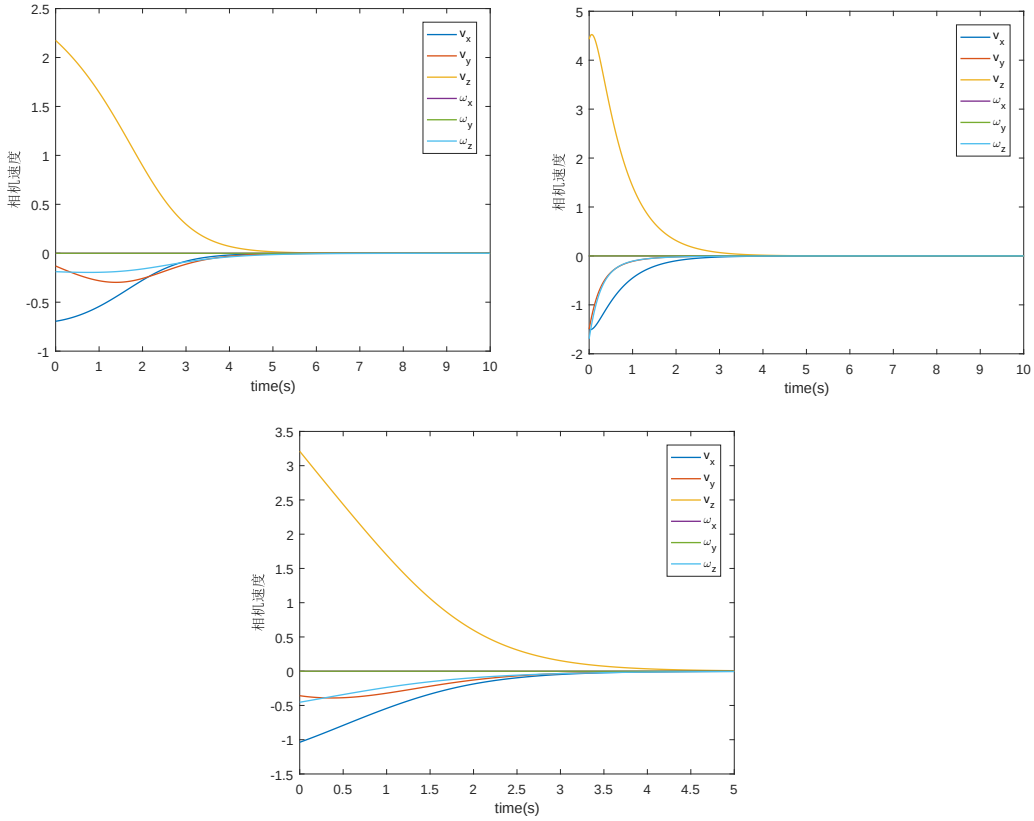

图 3: 相机速度

```
hold on
plot(t,camrow(:,2),'*g','linewidth',1);
hold on
plot(t,camrow(:,3),'-.r','linewidth',1);
xlabel('time(s)');
ylabel('横滚俯仰偏航角');
legend('R', 'P', 'Y');        %添加图例
saveas(gcf,'row1_pbvs','epsc');
figure(4);
plot(t,v_c,'linewidth',1);        %绘制相机速度
xlabel('time(s)');
ylabel('相机速度');
legend('v_{x}', 'v_{y}', 'v_{z}','\omega_{x}', '\omega_{y}', '\omega_{z
    }');        %添加图例,使用tex格式显示公式
saveas(gcf,'v1_pbvs','epsc');
```

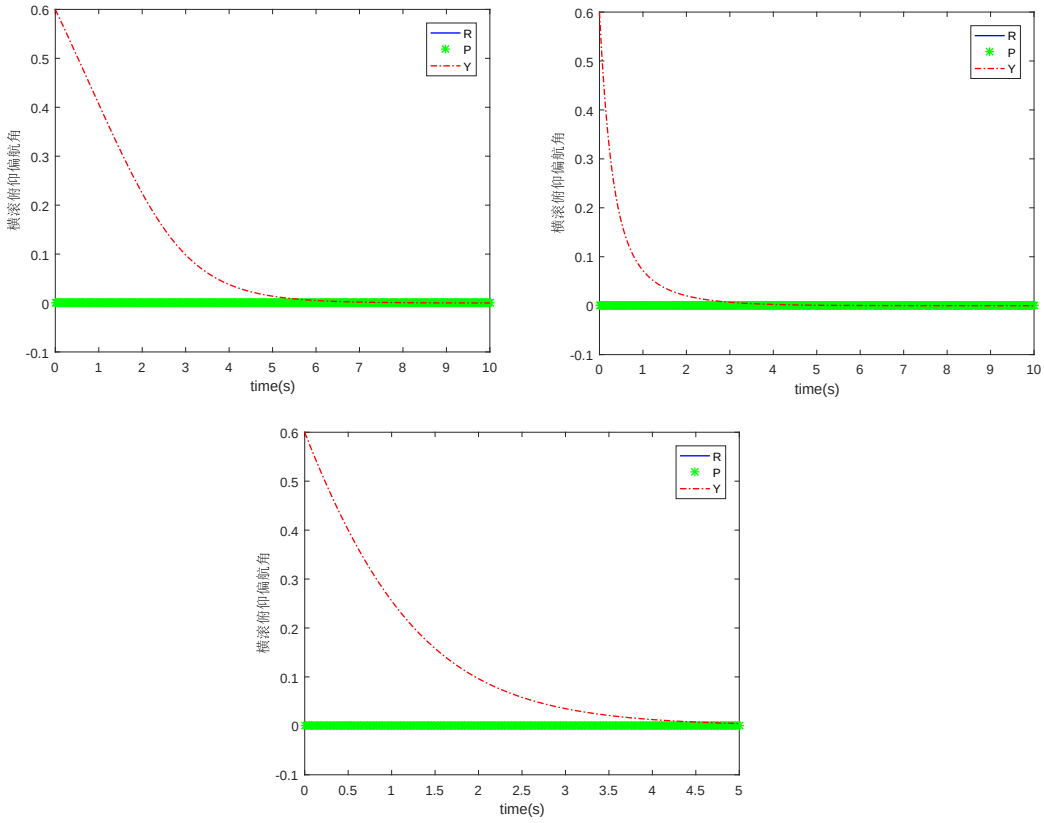

图 4: 俯仰横滚偏转角

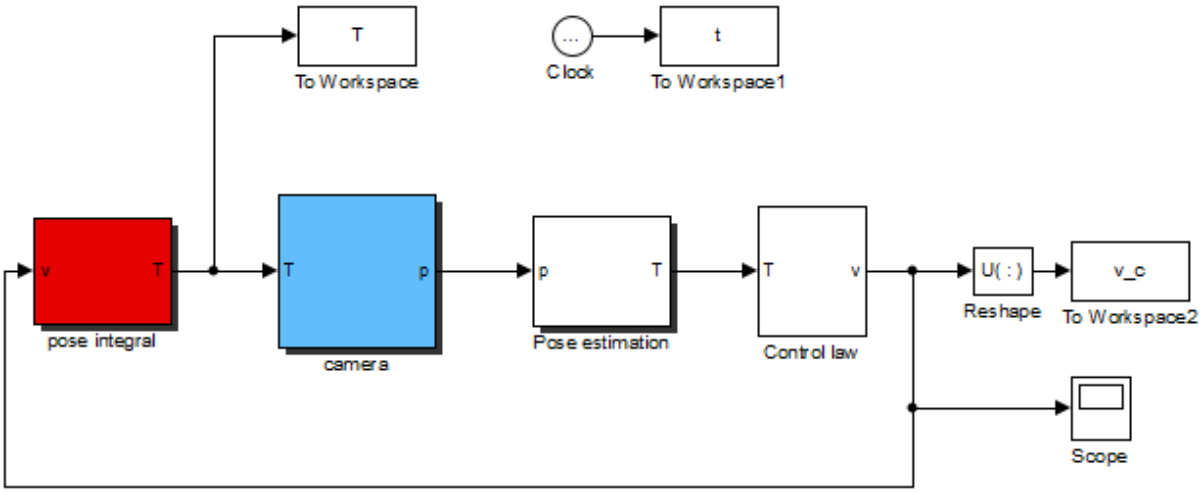

图 5: pbvs 仿真模型

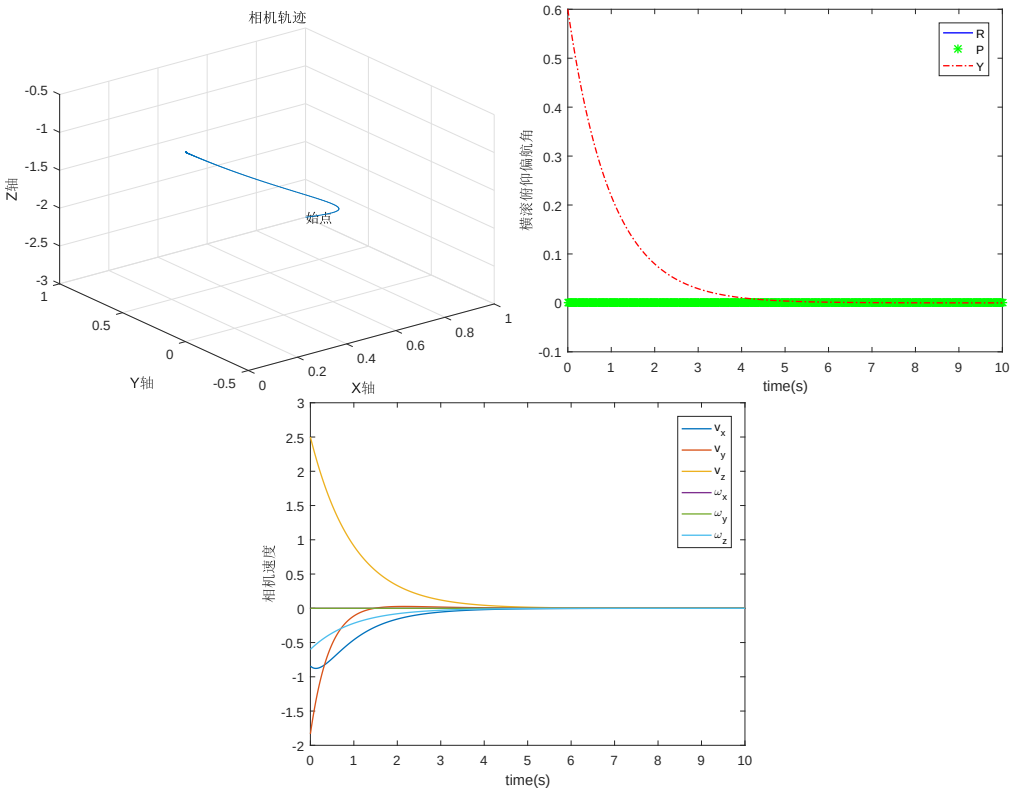
相机轨迹
Z轴
Y轴
X轴
始点
横滚俯仰偏航角
time(s)
R
P
Y
相机速度
time(s)
v_x
v_y
v_z
ω_x
ω_y
ω_z

图 6: 仿真结果